The Transformer Architecture (Transformer Architecture, Attention, Tokenization, & Decoding)

Niloy Ganguly

Professor

IIT Kharagpur





Attention Based Model: Transformer

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

https://www.youtube.com/watch?v=LWMzyfvuehA

Limitations of Pre-Attention Models

Key issues that motivated the development of Attention Mechanisms

01

Fixed-Length Context Bottleneck

Encoder-decoder models compressed entire sequences into one fixed-size vector — causing severe information loss for long inputs.

02

Vanishing & Exploding Gradients

RNNs suffered from gradients shrinking to zero or growing uncontrollably during backpropagation, blocking long-range learning.

03

Poor Long-Range Dependencies

Information from early tokens had to pass through many steps, getting diluted before reaching later positions in the sequence.

04

No Parallelism in Training

RNNs process tokens sequentially — each step depends on the previous — making them slow to train and difficult to scale.

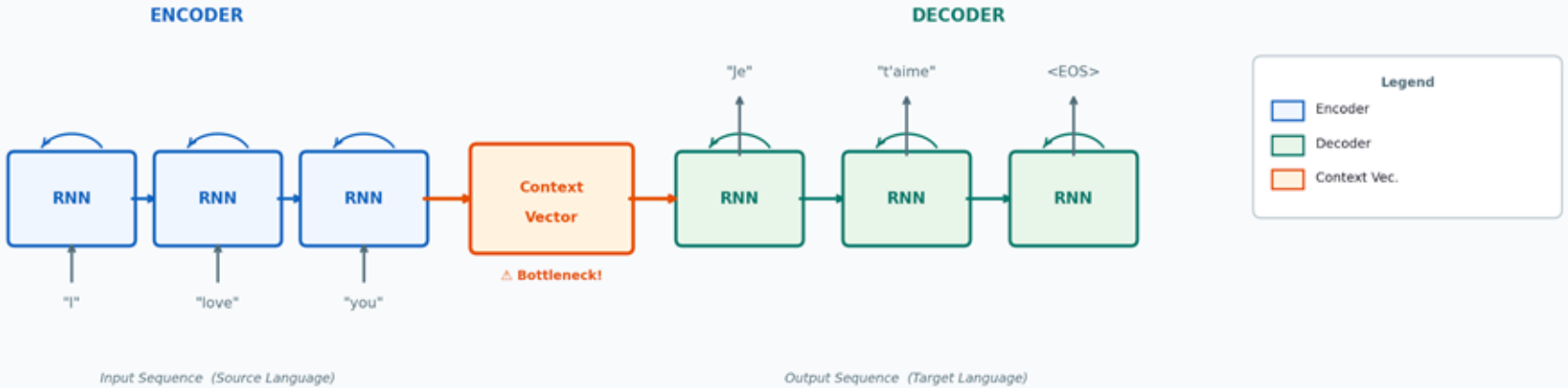
05

Uniform Treatment of Input

All input tokens were weighted equally when generating output, with no mechanism to selectively focus on what matters most.

Seq2Seq Model with RNN

How Encoder–Decoder RNNs process sequences — and where they fall short



① Encoder reads input tokens step-by-step, updating its hidden state at each token.

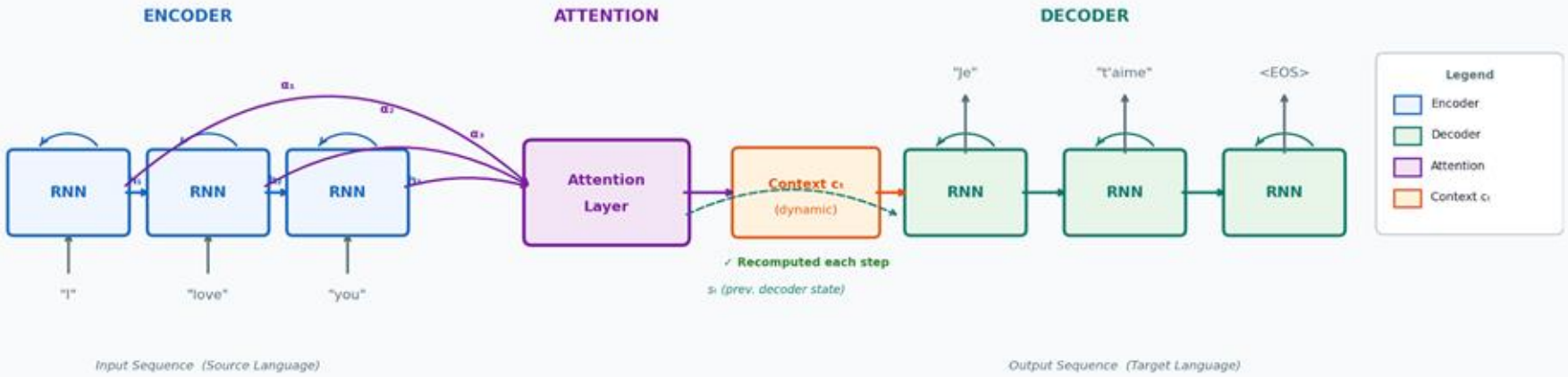
② The final hidden state becomes the Context Vector — a single fixed-size summary of all input.

③ The Decoder generates output tokens one at a time, conditioned only on this one context vector.

⚠ Key flaw: all input information is forced through one bottleneck vector, losing detail for long sequences.

Seq2Seq with Attention Layer

How the Attention mechanism transforms the Encoder–Decoder architecture



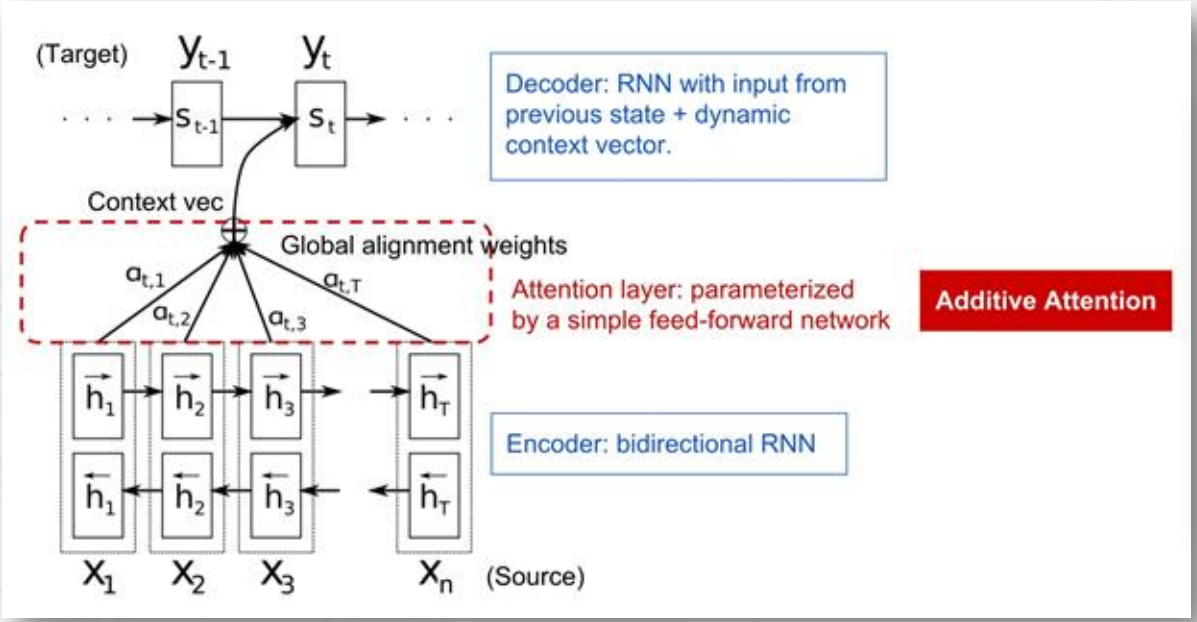
① All hidden states retained
Instead of a single final vector, all encoder hidden states $h_1 \ldots h_n$ are passed forward.

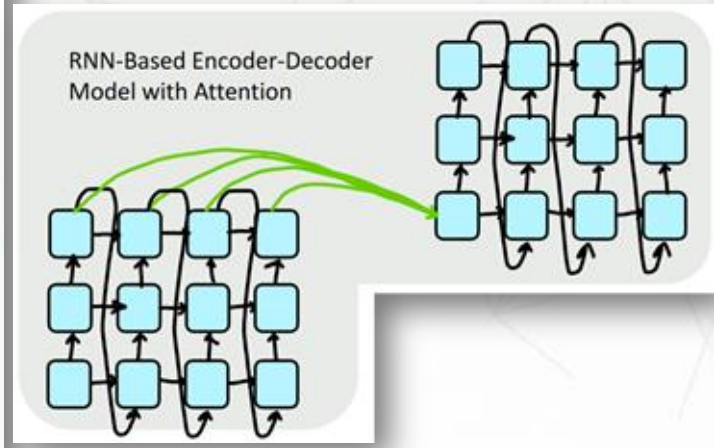
② Dynamic context per step
Attention computes a weighted sum (α_i) of encoder states for each decoder step.

③ Context c_t recomputed
A fresh context vector c_t is computed at every decoder timestep, focusing on what's relevant.

✓ Bottleneck eliminated
Long-range dependencies are captured directly — no information is lost through a single vector.

The Attention Idea





Attention treats each word's representation as a **query** to access and incorporate information from a **set of values**.

Bahdanau et al., 2015. Stanford NLP Course: CS224N

Why Attention Alone Wasn't Enough

Remaining limitations that led to Self-Attention and the Transformer architecture

Limitations of RNN + Attention

01 Still Sequential (RNN)
The RNN encoder/decoder processed tokens one at a time — no parallelisation across a sequence.

02 One-Directional Attention
Attention only ran decoder→encoder. Tokens within the encoder couldn't attend to each other.

03 Weak Long-Range in Encoder
Encoder states were still shaped by sequential RNN steps — early tokens got diluted before being attended to.

04 O(n) Information Path
Information still traveled through n recurrent steps to connect two distant tokens.

05 No Intra-Sequence Attention
Tokens couldn't ask 'what in this same sequence is relevant to me?' — the key idea of self-attention.

06 No Positional Structure
No built-in sense of position — any order had to be implicitly learned through the RNN.

led to

Transformer Solutions

Full Parallelism
RNNs removed entirely — all tokens processed simultaneously.

Self-Attention
Every token attends to every other token in the same sequence, bidirectionally.

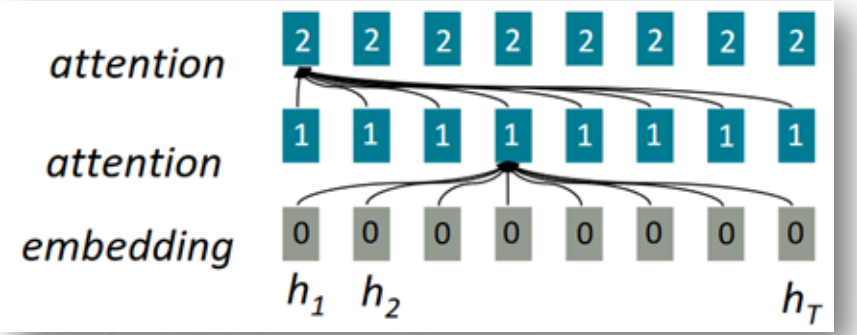
O(1) Dependency Path
Any two tokens connect directly in one attention step, regardless of distance.

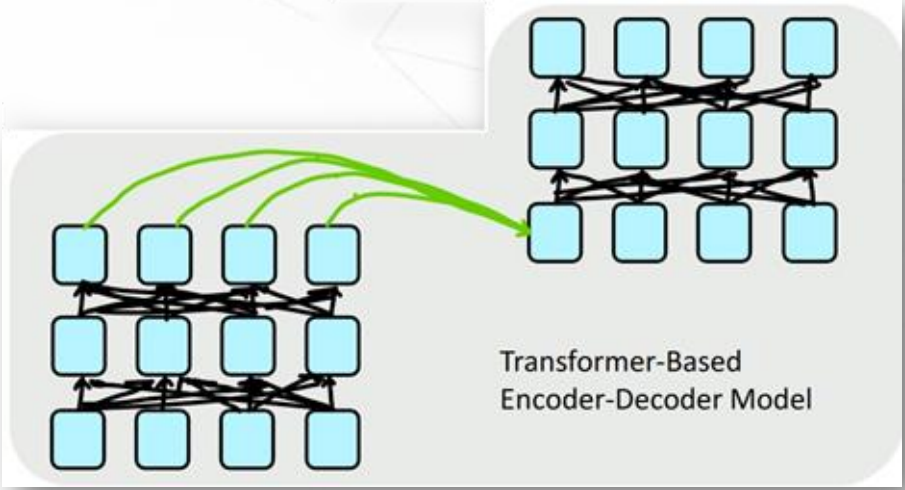
Multi-Head Attention
Multiple heads capture different relationship types in parallel.

Positional Encodings
Explicit position signals added to embeddings — order without recurrence.

Attention is All You Need
Vaswani et al. 2017: one unified architecture for encode, decode, and cross-attend.

The Self Attention Idea





Self-attention is encoder-encoder (or decoder-decoder) attention where each word attends to **each other word within the input** (or output).

Stanford NLP Course: CS224N

Attention vs self-attention

	Attention (Bahdanau)	Self-attention (Transformer)
Who attends to whom?	Decoder → Encoder — cross-sequence	Each word → every other word in same sequence — within-sequence
Used in	Seq2Seq translation (RNN-based)	Transformers — BERT, GPT, T5
Replaces	Fixed context vector bottleneck	Sequential LSTM entirely
Parallelizable?	Partially — encoder is parallel, decoder is sequential	Fully — no recurrence at all
Key formula	$c_t = \Sigma\ \alpha_i \cdot h_i$ $\alpha_i = \text{softmax}(FF(s_{t-1}, h_i))$	$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d}) \cdot V$

Attention asks "which source words matter for this output word?" — Self-attention asks "which words in this sentence matter for understanding each word?"

Attention Based Model: Transformer

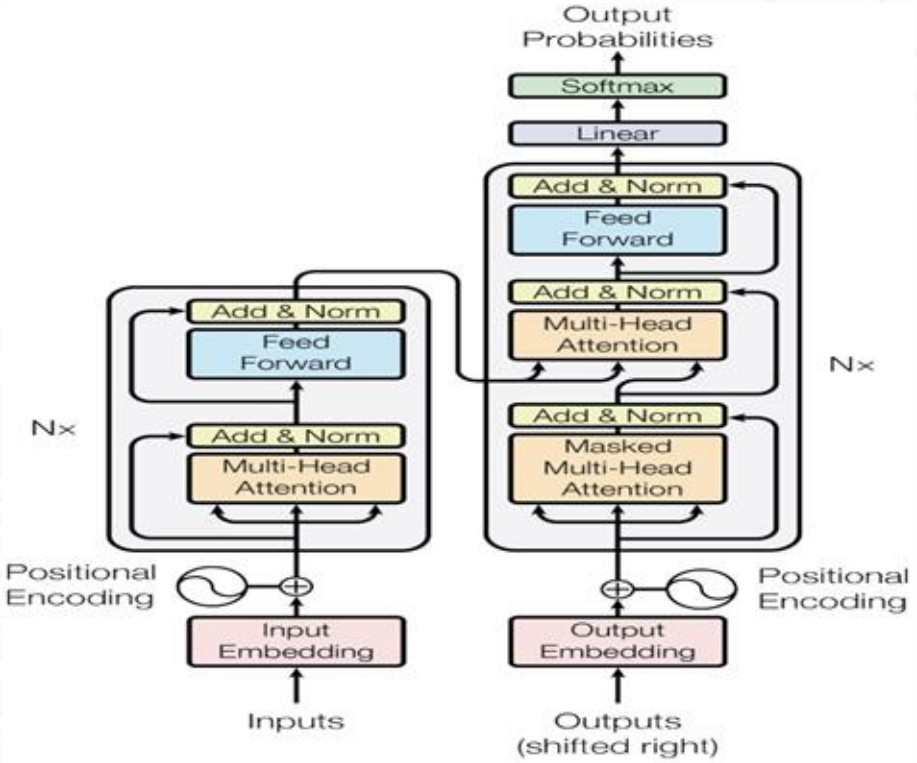


Figure 1: The Transformer - model architecture.

What is Self-Attention?

Each element looks at all others and decides how much to focus on each.

"The animal didn't cross the street because **it** was too tired."

👉 *What does "it" refer to?*

1 Compare

Compare it with every other word

2 Weight

Assign higher weight to animal

3 Understand

Use weights to build contextual meaning

Key Property

Every token can directly interact with every other token — no sequential propagation needed (unlike RNNs).

Long-range dependencies

Parallel computation

Dynamic context understanding

Query, Key, Value — Step by Step

"The animal didn't cross the street because it was too tired."

Q **Query**

What I am looking for

→ *it asks: who am I?*

K **Key**

What I represent

→ *animal says: I'm a noun*

V **Value**

What information I carry

→ *actual content to share*

Attention scores for "it":

Word	Key meaning	Score	Weight
animal	noun, living thing	**5.0**	**0.70**
tired	adjective	3.0	0.20
street	noun, location	1.0	0.05
others	...	small	~0

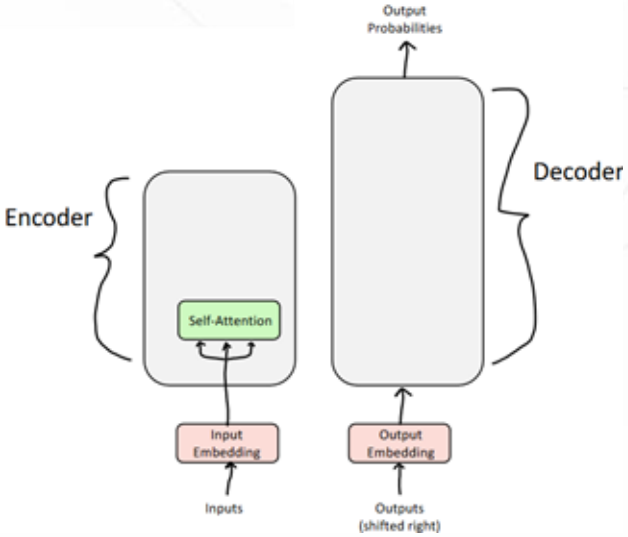
Output for "it":

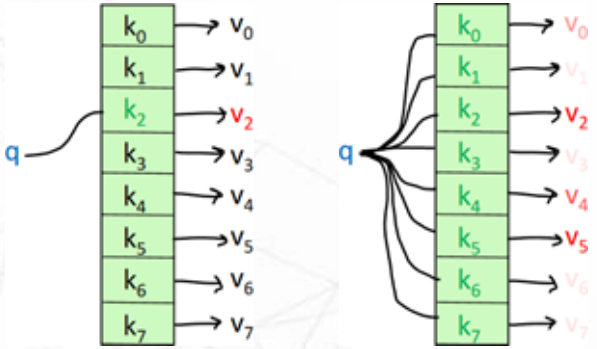
Output it = **0.70** × Vanimal + **0.20** × Vtired + ...

"it" becomes:
"something like animal, with context of being tired"

The (Self) Attention Mechanism

- Attention as a "fuzzy" or approximate hashtable
  - To look up a value, we compare a query against keys in a table
  - In a hashtable (shown on the bottom left):
    - Each query (hash) maps to exactly one key-value pair
  - In (self-)attention (shown on the bottom right)
    - Each query matches each key to varying degrees.
    - We return a sum of values weighted by the query-key match.





Recipe for Self-Attention in the Transformer Encoder

- Step 1: For each word x_i , calculate its **query**, **key**, and **value**.

$$q_i = W^Q x_i \quad k_i = W^K x_i \quad v_i = W^V x_i$$

- Step 2: Calculate attention score between **query** and **keys**.

$$e_{ij} = q_i \cdot k_j$$

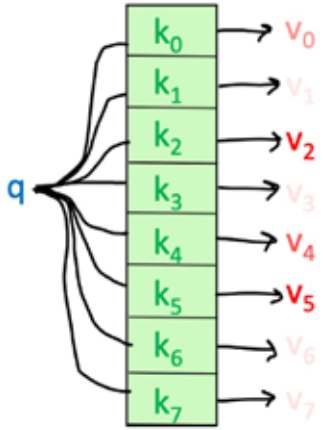
- Step 3: Take the softmax to normalize attention scores.

$$\alpha_{ij} = softmax(e_{ij}) = \frac{exp(e_{ij})}{\sum_k exp(e_{ik})}$$

- Step 4: Take a weighted sum of **values**.

$$Output_i = \sum_j \alpha_{ij} v_j$$

$$x_{(i+1)} = o_i$$



Recipe for (Vectorized) Self-Attention in the Transformer Encoder

- Step 1: With embeddings stacked in X, calculate **queries**, **keys**, and **values**.

$$Q = XW^Q \quad K = XW^K \quad V = XW^V$$

- Step 2: Calculate attention scores between **query** and **keys**.

$$E = QK^T$$

- Step 3: Take the softmax to normalize attention scores.

$$A = softmax(E)$$

- Step 4: Take a weighted sum of **values**.

$$Output = AV$$

$$Output = softmax\left(QK^T\right)V$$

Tiny Self-Attention — Step by Step

2 tokens: x_1 = "animal", x_2 = "it" · embedding dim d = 2

Input X

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Q = K = X (WQ=WK=I)

Scores = QK^T

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

softmax([1,0]) = [0.73, 0.27]
softmax([0,1]) = [0.27, 0.73]

WV (weights)

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

V = X × WV

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

→ **Attention A = softmax(Scores)**

$$\begin{bmatrix} 0.73 & 0.27 \\ 0.27 & 0.73 \end{bmatrix}$$

Output = A · V

"animal"

0.73·[1,1] + 0.27·[1,0]

= **[1.0, 0.73]**

"it"

0.27·[1,1] + 0.73·[1,0]

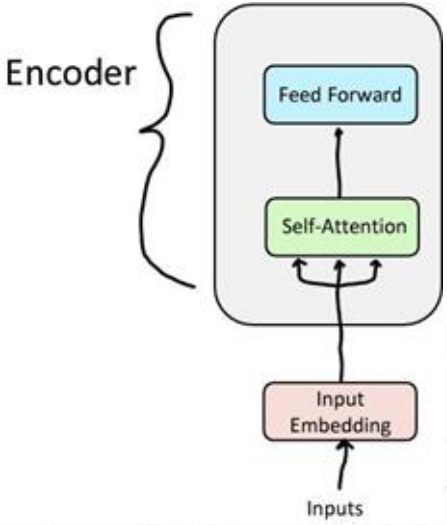
= [1.0, 0.27]

"it" has non-zero 2nd component → influenced by "animal" (which carries meaning [1,1])

Feed-forward Layer

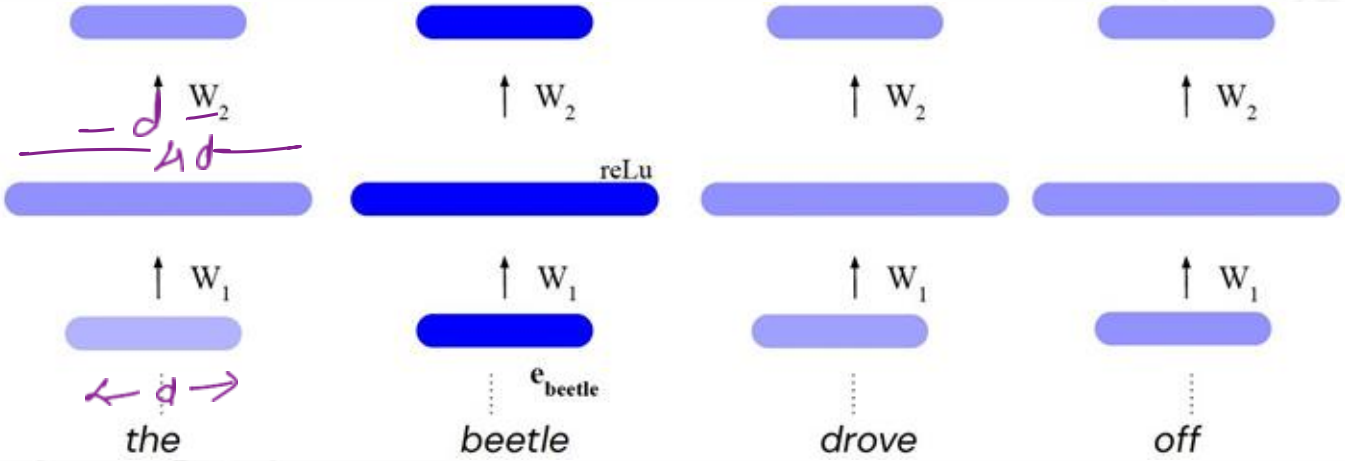
Problem: Since there are no element-wise non-linearities, self-attention is simply performing a re-averaging of the value vectors.

Easy fix: Apply a feedforward layer to the output of attention, providing non-linear activation (and additional expressive power).



Feed-forward Layer

$$\mathrm{FFN}(\mathbf{x}_i) = \mathrm{ReLU}(\mathbf{x}_i\mathbf{W}_1 + b_1)\mathbf{W}_2 + b_2$$



Attention = tokens talk to each other | FFN = each token thinks privately in 4d space | FFN parameters ≈ ⅔ of entire Transformer

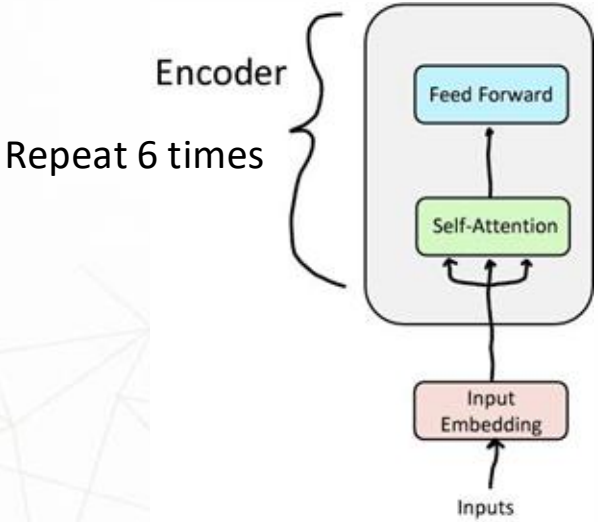
① Expand to 4d (W_1)
First linear layer projects each token from d → 4d dimensions. Creates a large private workspace for complex transformations.

② ReLU activation
Zeros out negative values, keeping only positive signals. Introduces non-linearity — without this, the network is just linear.

③ Compress back to d (W_2)
Second linear layer projects 4d → d. The refined, distilled representation is passed to Add & Norm and the next layer.

✓ Zero cross-token interaction
Unlike attention, FFN processes each token identically and independently. W_1 and W_2 are shared but applied position-wise.

How to make this work for deep networks?



Training Trick #1: Residual Connections
Training Trick #2: LayerNorm
Training Trick #3: Scaled Dot Product Attention

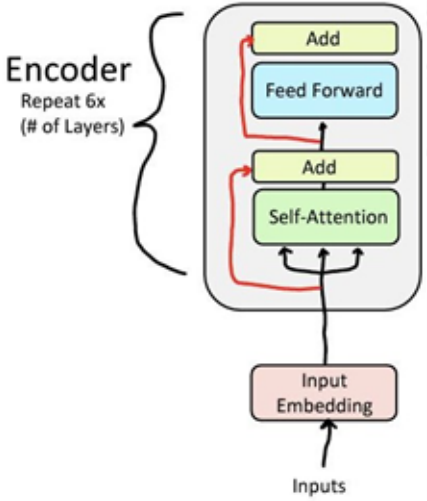
https://web.stanford.edu/class/cs224n
/

Training Trick #1: Residual Connections

- Residual connections are a simple but powerful technique from computer vision.
- Deep networks are surprisingly bad at learning the identity function!
- Therefore, directly passing "raw" embeddings to the next layer can actually be very helpful!

$$x_\ell = F(x_{\ell-1}) + x_{\ell-1}$$

- This prevents the network from "forgetting" or distorting important information as it is processed by many layers.

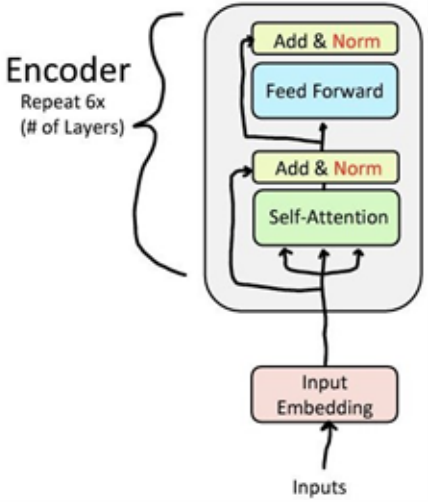


Training Trick #2: Layer Normalization

- **Problem:** Difficult to train the parameters of a given layer because its input from the layer beneath keeps shifting.
- **Solution:** Reduce variation by **normalizing** to zero mean and standard deviation of one within each **layer**.

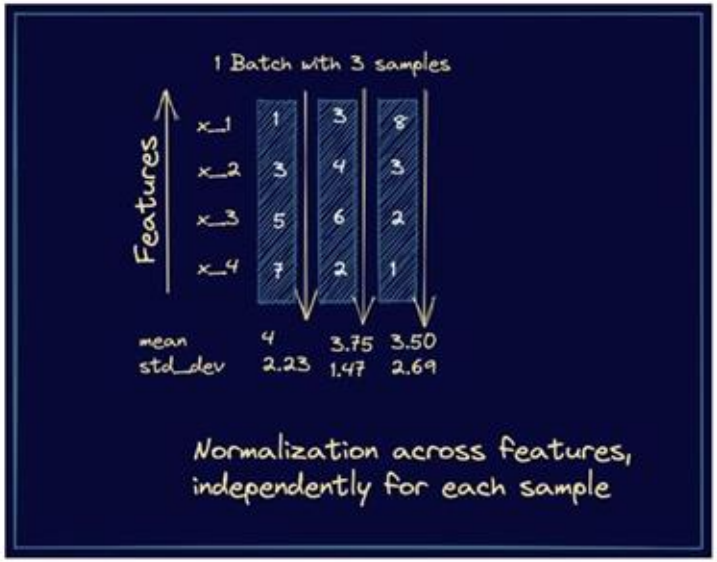
Mean: $\mu^l = \frac{1}{H}\sum_{i=1}^{H} a_i^l$ Standard Deviation: $\sigma^l = \sqrt{\frac{1}{H}\sum_{i=1}^{H}(a_i^l - \mu^l)^2}$

$$x^{\ell\prime} = \frac{x^\ell - \mu^\ell}{\sigma^\ell + \epsilon}$$

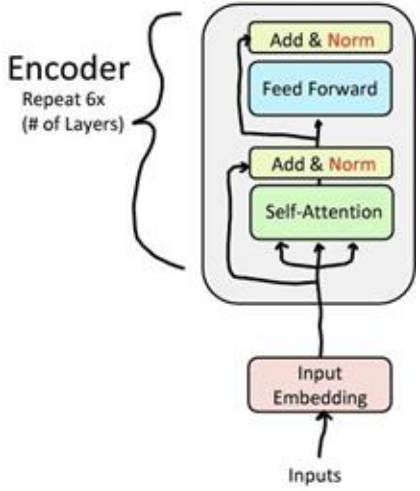


https://web.stanford.edu/class/cs224n/

Training Trick #2: Layer Normalization



An Example of How LayerNorm Works (Image by Bala Priya C, Pinecone)



Mean: $\mu^l = \frac{1}{H}\sum_{i=1}^{H} a_i^l$ Standard Deviation: $\sigma^l = \sqrt{\frac{1}{H}\sum_{i=1}^{H}(a_i^l - \mu^l)^2}$

$$x^{\ell\prime} = \frac{x^\ell - \mu^\ell}{\sigma^\ell + \epsilon}$$

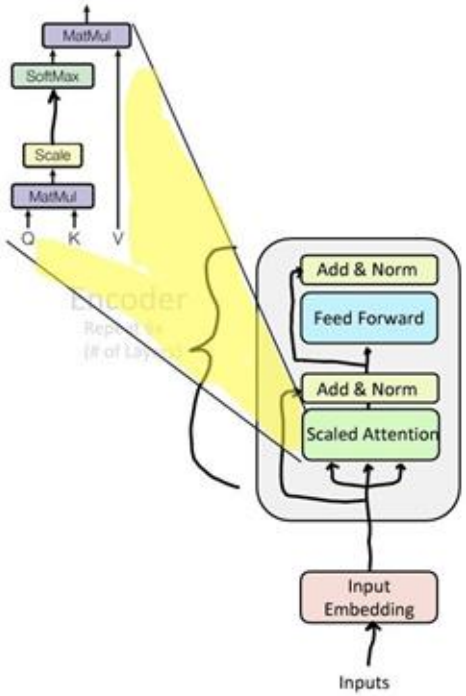
https://web.stanford.edu/class/cs224n/

Training Trick #3: Scaled Dot Product Attention

- After LayerNorm, the mean and variance of vector elements is 0 and 1, respectively. (Yay!)
- However, the dot product still tends to take on extreme values, as its variance scales with dimensionality d_k

Quick Statistics Review:

- Mean of sum = sum of means = $d_k * 0 = 0$
- Variance of sum = sum of variances = $d_k * 1 = d_k$
- To set the variance to 1, simply divide by $\sqrt{d_k}$!



https://web.stanford.edu/class/cs224n/

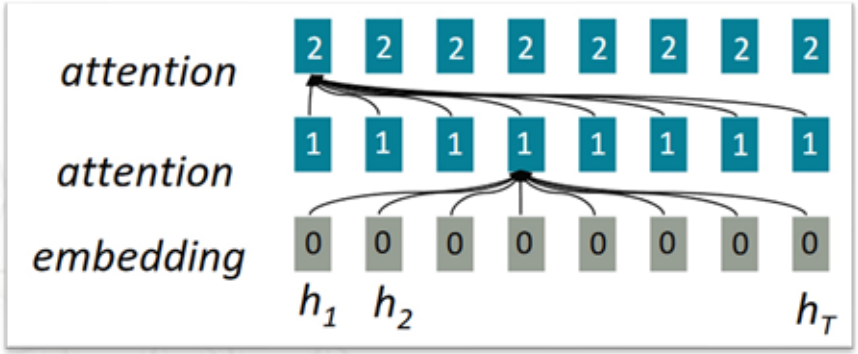
Training Trick #3: Scaled Dot Product Attention

- Assume that the components of q and k are independent random variables with mean 0 and variance 1.
- Then their dot product $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ has mean 0 and variance d_k.
- Hence the scaling ensures that the resultant dot product has mean 0 and variance 1.

Updated Self-Attention Equation:

$$Output = softmax\left(QK^T / \sqrt{d_k}\right) V$$

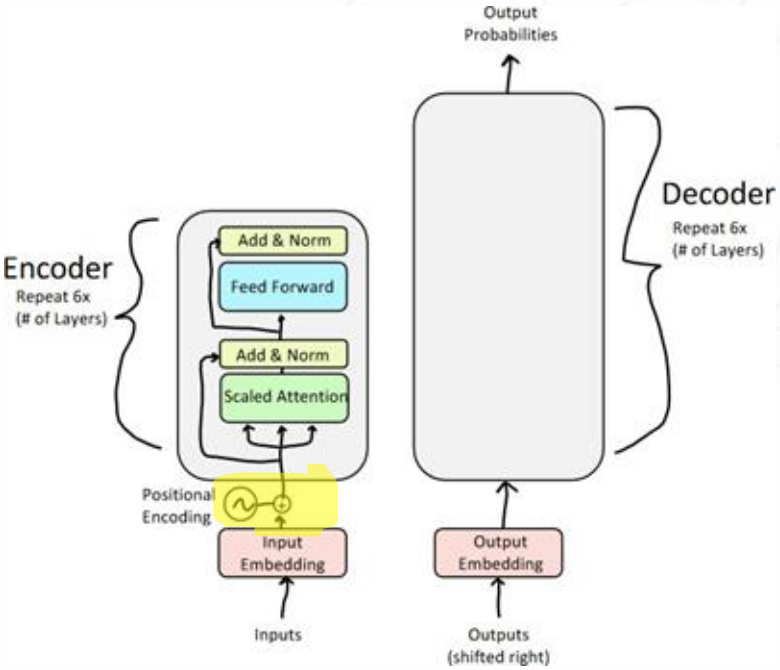
Positional Encoding



Sense of position is lost

$p_i \in \mathbb{R}^d$, for $i \in \{1,2,\ldots,T\}$ are position vectors

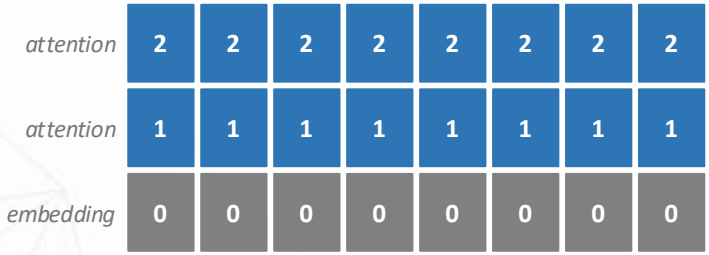
$$v_i = \tilde{v}_i + p_i$$
$$q_i = \tilde{q}_i + p_i$$
$$k_i = \tilde{k}_i + p_i$$



Positional Encoding

Sense of position is lost

Self-attention is permutation-invariant — token order is invisible to the model.
Positional encodings are added to token embeddings to inject sequence order.



$$PE(pos,\ 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos,\ 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

where **pos** = token position, **i** = dimension index, **d** = model dim

```
import math

class PositionalEncoding(nn.Module):
  def __init__(self, d=512, max_len=512):
    super().__init__()
    pe = torch.zeros(max_len, d)
    pos = torch.arange(0, max_len).float()
    pos = pos.unsqueeze(1)  # (max_len, 1)
    div = torch.exp(
      torch.arange(0, d, 2).float()
      * -(math.log(10000.0) / d))
    pe[:, 0::2] = torch.sin(pos * div)
    pe[:, 1::2] = torch.cos(pos * div)
    self.register_buffer('pe', pe.unsqueeze(0))

  def forward(self, x):
    # x: (B, seq_len, d_model)
    x = x + self.pe[:, :x.size(1)].to(x.device)
    return x
```

Injecting Order into Attention

```
import torch, math

def positional_encoding(max_len, d_model):
    pe = torch.zeros(max_len, d_model)
    pos = torch.arange(max_len).unsqueeze(1).float()    # (max_len, 1)
    div = torch.exp(torch.arange(0, d_model, 2).float()
                    * -(math.log(10000.0) / d_model))

    pe[:, 0::2] = torch.sin(pos * div)   # even dims: sine
    pe[:, 1::2] = torch.cos(pos * div)   # odd  dims: cosine
    return pe   # shape: (max_len, d_model)

# In the full model:
class TransformerEmbedding(nn.Module):
    def __init__(self, vocab_size, d_model, max_len=512):
        super().__init__()
        self.token_emb = nn.Embedding(vocab_size, d_model)
        self.pos_enc   = positional_encoding(max_len, d_model)

    def forward(self, x):
        tok = self.token_emb(x)                    # learned token
meaning
        pos = self.pos_enc[:x.size(1)].to(x.device) # fixed position
signal
        return tok + pos                            # element-wise sum!
```

Why sinusoidal?

Deterministic

No training needed for positions

Unique per position

Each pos has a unique fingerprint

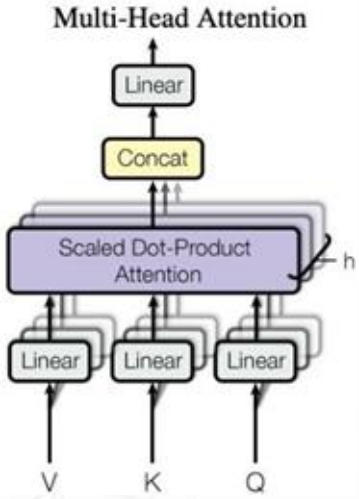
Relative distances

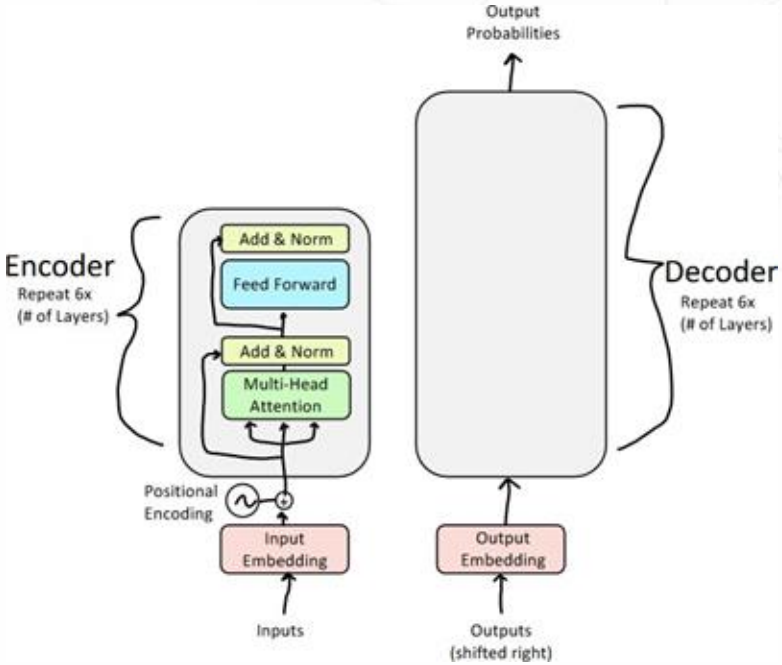
PE(pos+k) is a linear fn of PE(pos)

Generalizes

Works beyond training sequence lengths

Multi-Head Attention





Why Multiple Heads?

One attention head captures one type of relationship. Language has many.

Single Attention Head

One Q,K,V → captures only one type of relationship

→

Multi-Head Attention

h independent heads → captures h different relationship types

H1 — **Coreference**

Example: "it" → "animal"

H2 — **Verb–Object**

Example: "cross" → "street"

H3 — **Semantic sim.**

Example: "tired" → "animal"

H4 — **Positional**

Example: nearby tokens

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1 \dots \text{head}_h) \cdot \mathbf{W^o} \qquad \text{head}_i = \text{softmax}(Q_i K_i^T / \sqrt{d_k})\, V_i$$

2-Head Attention — Concrete Calculation

x_1="animal" x_2="it" · X = [[1,0],[0,1]] · Each head dim = 1

HEAD 1

→ focuses on "animal" (coreference)

W^Q=[[1,1]^T] W^K=[[1,0]^T] W^V=[[2,0]^T]

Q=[1,1]^T K=[1,0]^T V=[2,0]^T

QK^T = [[1,0],[1,0]]

A = softmax → [[0.73,0.27],[0.73,0.27]]

Output: animal=[1.46] it=[1.46]

HEAD 2

→ focuses on self / context

W^Q=[[0,1]^T] W^K=[[0,1]^T] W^V=[[0,3]^T]

Q=[0,1]^T K=[0,1]^T V=[0,3]^T

QK^T = [[0,0],[0,1]]

A = softmax → [[0.5,0.5],[0.27,0.73]]

Output: animal=[1.5] it=[2.19]

Concat → **"animal":[1.46, 1.5]** | **"it":[1.46, 2.19]** → multiply by W^O for final output

Head 1: Computing Q, K, V

Projection matrices WQ, WK, WV → Q, K, V vectors for each token

X = **[[1,0],[0,1]]** x_1='animal'=[1,0] x_2='it'=[0,1] Head dim = 1

PROJECTION MATRICES (each is 2×1 — projects d=2 embedding to 1-dimensional head space)

$WQ^{(1)}$ =

$$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Both dimensions equally projects to combined space

$WK^{(1)}$ =

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Only 1st dimension picks up 'animal' signal

$WV^{(1)}$ =

$$\begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

Scaled 1st dim (×2) amplifies animal info

COMPUTED Q, K, V

$Q^{(1)} = X \cdot WQ^{(1)}$ animal=$[1,0]\cdot[1,1]^T = 1$
it=$[0,1]\cdot[1,1]^T = 1$

$Q^{(1)} = [1, 1]^T$

$K^{(1)} = X \cdot WK^{(1)}$ animal=$[1,0]\cdot[1,0]^T = 1$
it=$[0,1]\cdot[1,0]^T = 0$

$K^{(1)} = [1, 0]^T$

$V^{(1)} = X \cdot WV^{(1)}$ animal=$[1,0]\cdot[2,0]^T = 2$
it=$[0,1]\cdot[2,0]^T = 0$

$V^{(1)} = [2, 0]^T$

Head 1: Scores, Softmax & Attention Matrix

$Q^{(1)} = [1,1]^T \cdot K^{(1)} = [1,0]^T$

STEP 3 — Score Matrix = QK^T

$Q^{(1)} \cdot (K^{(1)})^T$

$Q^{(1)}$ 2×1 $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ × $(K^{(1)})^T$ 1×2 $\begin{bmatrix} 1 & 0 \end{bmatrix}$ = Scores 2×2 $\begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}$

Row 1 (animal): animal attends to animal: 1·1+0·0 = **1.0** animal attends to it: 1·0+0·1 = **0.0**

Row 2 (it): it attends to animal: 1·1+0·0 = **1.0** it attends to it: 1·0+0·1 = **0.0**

Key insight: Both rows are [1, 0] → both tokens attend equally to animal!

STEP 4 — Softmax → Attention Matrix $A^{(1)}$

$$\text{softmax}([x_1, x_2]) = [e^{x1}/(e^{x1}+e^{x2}),\ e^{x2}/(e^{x1}+e^{x2})]$$

Row 1: softmax([1, 0]) $e^1 = 2.718$ $e^0 = 1.000$ sum = 3.718

[2.718/3.718, 1.000/3.718] = [0.73, 0.27]

Row 2: softmax([1, 0]) → same! (both rows of score matrix are [1,0])

[0.73, 0.27]

∴ **Attention Matrix $A^{(1)}$ =**

[[0.73, 0.27], [0.73, 0.27]] → both rows identical!

Head 1: Output = $A^{(1)} \cdot V^{(1)}$

$A^{(1)} = [[0.73,0.27],[0.73,0.27]] \cdot V^{(1)} = [2, 0]^T$

MATRIX MULTIPLICATION

$A^{(1)}$		$V^{(1)}$		$O^{(1)}$
[0.73 0.27]	·	[2]	=	**[1.46]**
[0.73 0.27]		[0]		**[1.46]**

Row-by-Row Calculation:

Row 1 — "animal":

$0.73 \times 2 + 0.27 \times 0 = 1.46$

Row 2 — "it":

$0.73 \times 2 + 0.27 \times 0 = 1.46$

Both tokens output 1.46 — Head 1 enforces coreference (both attend to 'animal')

Interpretation

Why both outputs = 1.46?

Attention matrix rows are identical [0.73, 0.27]
→ Both "animal" and "it" use the same mixing recipe over $V^{(1)} = [2, 0]^T$

What does 1.46 represent?

Value from 'animal' (V=2) weighted by 0.73
Value from 'it' (V=0) weighted by 0.27
→ Mostly 'animal' signal

Coreference effect:

After Head 1, both tokens carry the same representation heavily influenced by 'animal' → coreference resolved

$O^{(1)} = [1.46, 1.46]^T$ — both tokens equally 'animal-aware'

Head 2: Full Computation — Self / Context Attention

$WQ = WK = [0,1]^T$ · $WV = [0,3]^T$ → Head 2 focuses on the second dimension

Steps 1–2: Projections → Q, K, V

$WQ^{(2)} = [0,1]^T$

$WK^{(2)} = [0,1]^T$

$WV^{(2)} = [0,3]^T$

$Q^{(2)} = X \cdot WQ^{(2)}$

animal=$[1,0]\cdot[0,1]^T=0$

it=$[0,1]\cdot[0,1]^T=1$

→ $Q^{(2)}=[0,1]^T$

$K^{(2)}=[0,1]^T$ (same)

$V^{(2)}=[0,3]^T$

animal V=0, it V=3

Steps 3–4: Scores → Softmax

$QK^T = [0,1]^T \cdot [0,1] =$

$$\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

Row 1 (animal):

softmax([0,0])

$e^0=1$, $e^0=1$, sum=2

→ [0.50, 0.50]

Row 2 (it):

softmax([0,1])

$e^0=1$, $e^1=2.718$, sum=3.718

→ [0.27, 0.73]

$A^{(2)}$ = [[0.5,0.5],[0.27,0.73]]

Step 5: Output = $A^{(2)} \cdot V^{(2)}$

$V^{(2)} = [0, 3]^T$

(animal→0, it→3)

Row 1 "animal":

0.50×0 + 0.50×3

= 1.50

animal gets equal mix of self (0) and it (3)

Row 2 "it":

0.27×0 + 0.73×3

= 2.19

it attends mostly to itself → larger value

$O^{(2)} = [1.50, 2.19]^T$ — 'it' stronger (self-aware)

Combining Heads: Concatenation & Final Output

$O^{(1)}$ — Head 1 output

Dim = 1 per token

"animal" **[1.46]**

"it" **[1.46]**

coreference signal

⊕

$O^{(2)}$ — Head 2 output

Dim = 1 per token

"animal" **[1.50]**

"it" **[2.19]**

self/context signal

→

Concat($O^{(1)}$, $O^{(2)}$) → 2D output

Dim = 1+1 = 2 per token

"animal" **[1.46, 1.50]**

"it" **[1.46, 2.19]**

→ multiply by W^O for final output

Token	H1 Output	H1 Meaning	H2 Output	H2 Meaning	Final Representation
"animal"	**[1.46]**	Attends to itself (main noun)	**[1.50]**	Equal mix → self-aware	**Knows it is the coreference target**
"it"	**[1.46]**	Attends to animal (coreference!)	**[2.19]**	Attends to itself (identity)	**"it" = animal + context — resolved!**

Head 1 vs Head 2 — Side-by-Side Comparison

Same input X, different projection matrices → different attention patterns → complementary information

	HEAD 1 (Coreference)	HEAD 2 (Self/Context)
WQ	$[1,1]^T$ — both dims equally	$[0,1]^T$ — 2nd dim only
WK	$[1,0]^T$ — 1st dim only	$[0,1]^T$ — 2nd dim only
WV	$[2,0]^T$ — amplify 1st dim	$[0,3]^T$ — amplify 2nd dim
Scores	QK^T = [[1,0],[1,0]] (both rows same)	QK^T = [[0,0],[0,1]] (rows differ)
Attn A	[[0.73,0.27],[0.73,0.27]] (rows identical)	[[0.50,0.50],[0.27,0.73]] (rows differ)
Output	animal=[1.46] it=[1.46] (same!)	animal=[1.50] it=[2.19] (differ)
Learns	Coreference: it → animal	Self-awareness: it → itself

Identical Attention (Head 1)

When projection matrices create equal scores for all tokens, every token attends with the same pattern → enforces group-level coreference.

Differential Attention (Head 2)

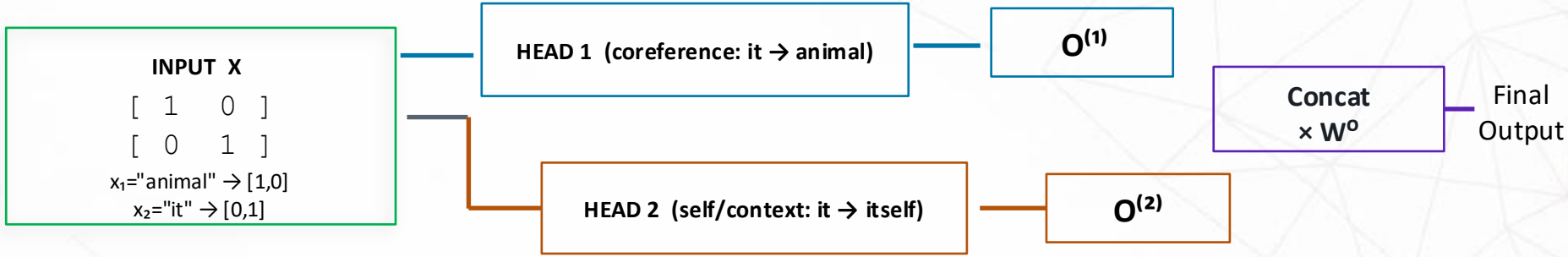
When projection matrices separate tokens, each attends differently → preserves individual identity while tracking context.

Why Both Together Are Powerful

Head 1 links 'it' to 'animal' (coreference). Head 2 preserves 'it' as a separate entity. Together: 'it' = 'animal' + distinct context.

2-Head Attention — Full Walkthrough

x_1 = "animal" · x_2 = "it" · Embedding dim d = 2 · 2 heads, each dim = 1



H1 What Head 1 learns

WQ projects both tokens to same space $[1,1]^T$

WK focuses only on first dimension $[1,0]^T$

→ animal gets score 1, it gets score 1

→ Both attend to animal (coreference)

H2 What Head 2 learns

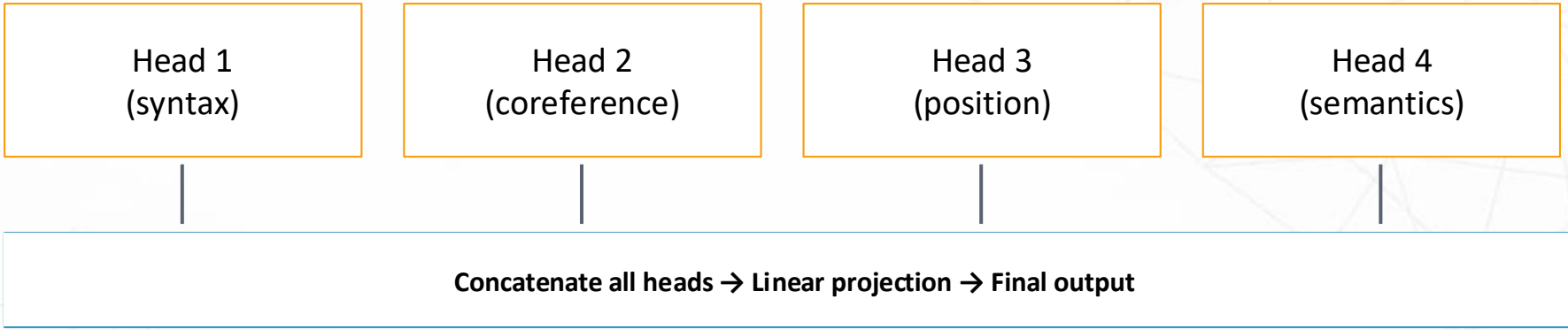
WQ focuses on second dimension $[0,1]^T$

WK same projection $[0,1]^T$

→ animal gets score 0, it gets score 1

→ it attends to itself (context-aware)

Multi-Head Attention



PyTorch

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model=512, n_heads=8):
        super().__init__()
        self.h = n_heads
        self.d_k = d_model // n_heads          # 64 per head
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def split_heads(self, x, B):
        return x.view(B, -1, self.h, self.d_k).transpose(1, 2)
```

```
def forward(self, Q, K, V):
        B = Q.size(0)
        Q = self.split_heads(self.W_q(Q), B)  # (B, h, T, d_k)
        K = self.split_heads(self.W_k(K), B)
        V = self.split_heads(self.W_v(V), B)
        scores = (Q @ K.transpose(-2,-1)) / self.d_k**0.5
        out = F.softmax(scores, dim=-1) @ V # 8 parallel attention heads
        out = out.transpose(1,2).contiguous().view(B, -1, self.h *
self.d_k)
        return self.W_o(out) # project back to d_model
```

The Complete Transformer Stack

PyTorch — Full Model

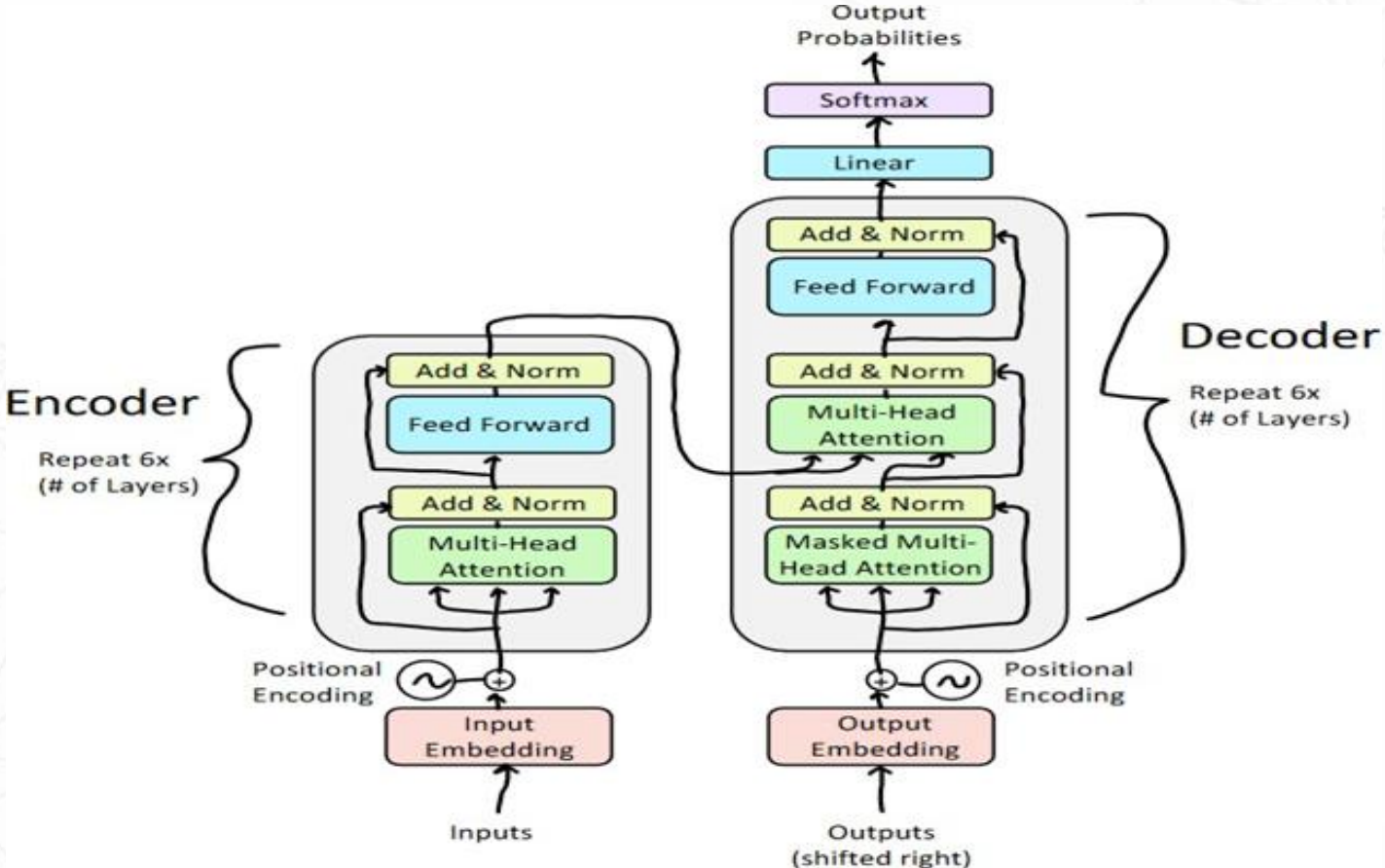
```
class Transformer(nn.Module):
    def __init__(self, vocab_size=50000, d_model=512,
                 n_heads=8, n_layers=6, d_ff=2048, max_len=512):
        super().__init__()
        self.embedding = TransformerEmbedding(vocab_size, d_model, max_len)
        self.encoder   = nn.ModuleList(
            [TransformerEncoderLayer(d_model, n_heads, d_ff) for _ in range(n_layers)])
        self.fc_out    = nn.Linear(d_model, vocab_size)

    def forward(self, src):
        x = self.embedding(src)          # token + position
        for layer in self.encoder:
            x = layer(x)                 # 6× self-attention + FFN
        return self.fc_out(x)            # project to vocab

# GPT-2 style: 12 layers, 12 heads, d_model=768, d_ff=3072
model = Transformer(vocab_size=50257, d_model=768,
                    n_heads=12, n_layers=12, d_ff=3072)
params = sum(p.numel() for p in model.parameters())
print(f"Parameters: {params/1e6:.0f}M")   # ~117M (GPT-2 small)
```

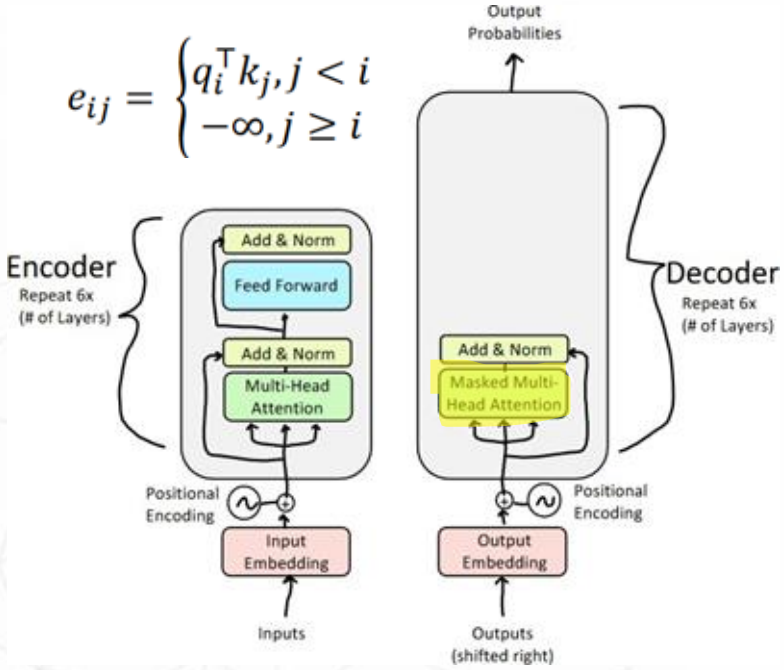


The Transformer (Decoder)

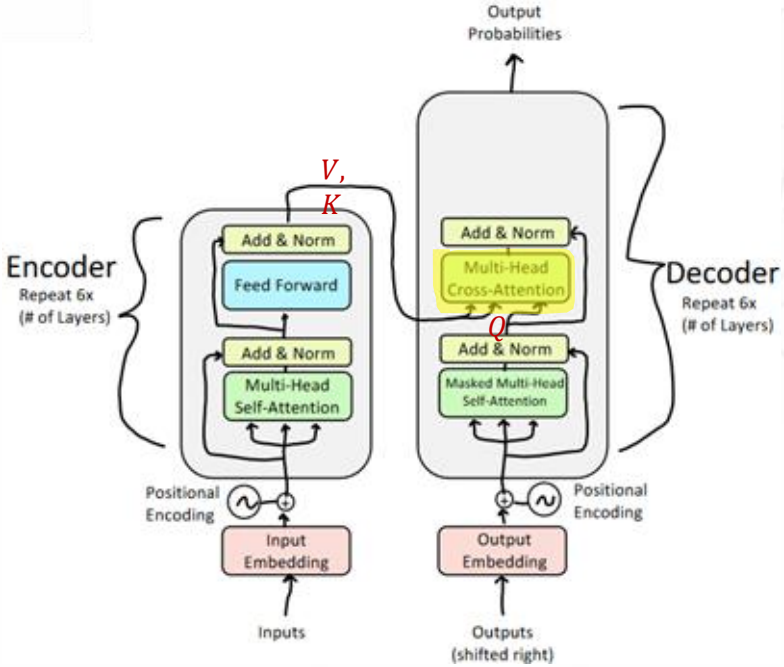


Decoder Design

$$e_{ij} = \begin{cases} q_i^T k_j, j < i \\ -\infty, j \geq i \end{cases}$$



Masked Multi-head Attention



Encoder-Decoder Multi-head Attention

The Transformer Decoder: Overview

Three sub-layers per decoder block:

(i) Masked Multi-Head Self-Attention

(ii) Cross-Attention (Encoder–Decoder)

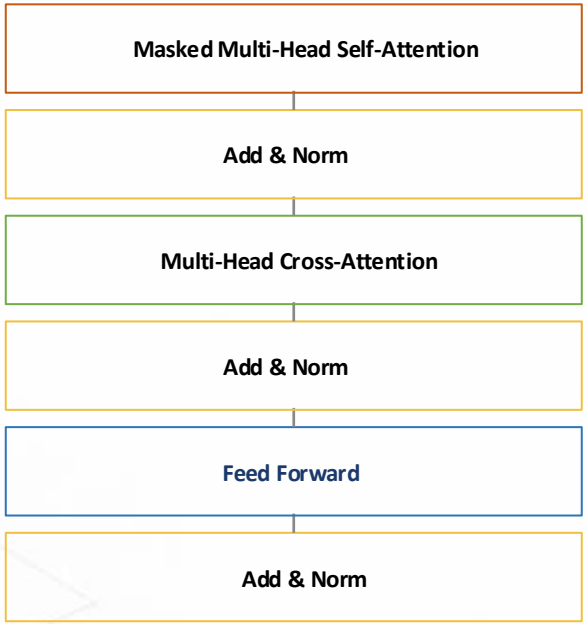
(iii) Feed-Forward Network (FFN)

Each sub-layer wrapped with:

Residual Connection + Layer Norm

Repeated N times (N = 6 in original paper)

Final Linear + Softmax → token probability



Python

```
import torch.nn as nn

class DecoderLayer(nn.Module):
  def __init__(self, d=512, h=8):
    super().__init__()
    self.self_attn = MaskedMHA(d,h)
    self.cross_attn = CrossMHA(d,h)
    self.ffn = FFN(d)
    self.norm1 = nn.LayerNorm(d)
  def forward(self, x, mem):
```

The Complete Transformer Decoder — Summary

Masked Self-Attention

Causal mask: attend only to past tokens. Prevents future leakage during training.

Cross-Attention

Q from decoder, K/V from encoder. Dynamically reads source context.

Feed-Forward (FFN)

Two-layer MLP with ReLU, applied per token. d_ff = 4 x d_model.

Add & Norm

Residual + LayerNorm after every sub-layer. Stabilises training.

Positional Encoding

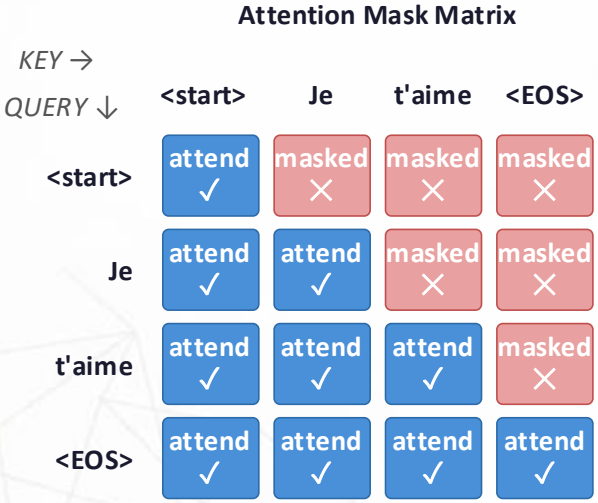
Sine/cosine vectors injected into embeddings. Order-aware attention.

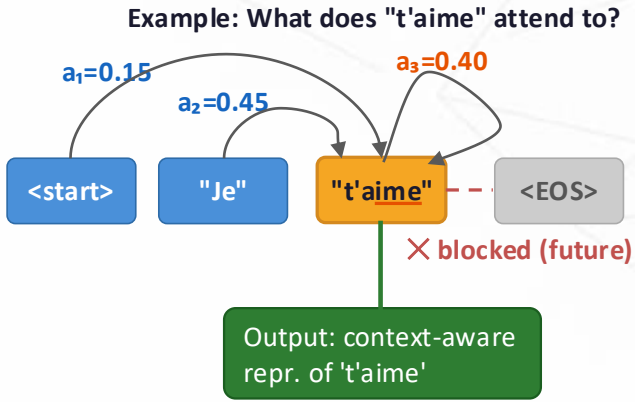
Autoregressive Loop

One token per step. Append and repeat until <EOS> or max length.

Masked Self-Attention

Every token attends to every past token — but is strictly blocked from seeing the future





✓ Can attend (past + self)
Score computed normally

✗ Masked (future position)
Score set to −∞ before softmax

Key Insight: The mask enforces left-to-right generation during training while still allowing full parallelism across the sequence.

① Self-attention within decoder
Each output token computes attention over all tokens in the same sequence — not the encoder.

② The mask blocks the future
Positions to the right are set to −∞ before softmax, so they contribute zero weight — model cannot cheat.

③ Triangular attention pattern
The result is a lower-triangular matrix: token 1 sees only itself, token 4 sees tokens 1–4.

✓ Parallel training, causal output
Mask lets the whole sequence train in parallel while preserving left-to-right generation order.

Masked Self-Attention — Step by Step

How Q, K, V matrices and the mask combine to produce context-aware representations

Each step: ONE query token — repeated in parallel for ALL tokens simultaneously

Input Embedding + Position	Linear Projection Q, K, V	Attention Scores $Q \cdot K^T / \sqrt{d}$	Apply Mask (−∞ future)	Softmax → Weights $\alpha_1\ \alpha_2\ \alpha_3$ ...	Weighted Sum of Values V
Token vectors with position info baked in	Three learned weight matrices project input	Dot product of query & keys scaled by √d	Future positions set to −inf before softmax	Converts scores to probabilities (sum = 1)	Context vector combining all attended tokens
Same input serves Q, K, V	*Q = what I want K = what I have V = what I give*	*High score = more relevant token*	*−inf → 0 after softmax = invisible*	*Weights show what was attended*	*Richer repr. than input alone*

Q Query — What am I looking for?
The current token projects itself into Q-space asking: which other tokens are relevant to me right now?

K Key — What do I offer?
Every past token projects into K-space, advertising what information it contains for others to use.

V Value — What do I give?
Once relevance is scored, V holds the actual content each token contributes to the output representation.

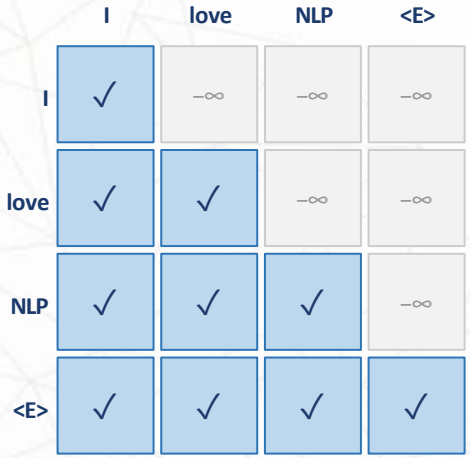
✓ Multi-head = multiple views
In practice, 8–16 heads run in parallel — each learning different Q/K/V projections to capture different patterns.

Masked Multi-Head Self-Attention

Why masking?

- During training, the full target sequence is available.
- Without a mask, decoder could attend to future tokens (cheating).
- Causal mask sets future scores to -inf => softmax weight ~ 0.

$$e_{ij} = q_i^T k_j \quad (j < i)$$
$$e_{ij} = -\inf \quad (j >= i)$$



Python

```
import torch, torch.nn.functional as F

def causal_mask(seq_len):
    # Lower-triangular mask: True = allowed
    mask = torch.tril(torch.ones(seq_len, seq_len))
    return mask == 0  # True = masked out

def masked_attention(Q, K, V, mask):
    d_k = Q.shape[-1]
    scores = torch.matmul(Q, K.transpose(-2,-1))
    scores = scores / d_k ** 0.5

    # Apply causal mask
    if mask is not None:
        scores = scores.masked_fill(
            mask, float('-inf'))

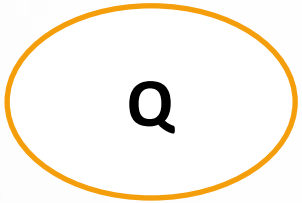
    weights = F.softmax(scores, dim=-1)
    return torch.matmul(weights, V)

# Example usage
mask = causal_mask(5)  # shape: (5,5)
```

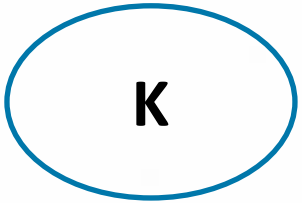
Cross-Attention

Let the decoder ask questions of the encoder

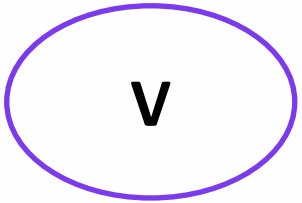
Cross-Attention: Q, K, V



Query
(Decoder)



Key
(Encoder)



Value
(Encoder)

=

$$\text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

Scaled Dot-Product Attention

PyTorch

```
import torch, torch.nn.functional as F

def cross_attention(Q, K, V):
    # Q: (batch, tgt_len, d_k) – from decoder
    # K, V: (batch, src_len, d_k) – from encoder
    d_k = Q.shape[-1]

    scores = torch.bmm(Q, K.transpose(1, 2))# (batch, tgt_len, src_len)
    scores = scores / d_k**0.5 # scale to prevent vanishing grad

    weights = F.softmax(scores, dim=-1) # attention distribution over src
    output  = torch.bmm(weights, V)     # weighted sum of values
    return output, weights
```

```
# Example: Translating EN → FR
# "The cat sat" → encoder hidden states (K, V)
# "Le chat"     → decoder states so far (Q)
# English tokens
src_hidden = torch.randn(1, 3, 512)
# English tokens
tgt_hidden = torch.randn(1, 2, 512)
out, attn_weights = cross_attention(tgt_hidden,
src_hidden, src_hidden)

print(attn_weights.shape)

# (1, 2, 3) – each FR token attends to
3 EN tokens
```

What is Cross-Attention?

The decoder looks at encoder outputs to decide what source information to use.

EN: "The cat sat on the mat." → FR: "**Le chat** s'est assis sur le tapis."

👉 *Which English words should "chat" attend to?*

1 Query

Decoder token asks: what source info do I need?

2 Match

Score query against each encoder key

3 Aggregate

Weighted sum of encoder values for the decoder

Key Difference from Self-Attention

Q comes from the DECODER, while K and V come from the ENCODER — two different sequences interact, unlike self-attention where all three come from the same sequence.

Two-sequence bridging

Encoder-decoder alignment

Used in translation & generation

Cross-Attention: Query, Key, Value — Step by Step

EN: "The cat sat on the mat." → FR: "Le chat s'est assis sur le tapis."

Q **Query**

What I am looking for

→ *"Le chat" asks: which EN words describe me?*

K **Key**

What I represent

→ *each EN token says: here is what I represent*

V **Value**

What information I carry

→ *encoder hidden state to blend*

Attention scores for "chat" (FR decoder):

Word	Key meaning	Score	Weight
cat	subject noun	**5.0**	**0.70**
sat	past verb	3.0	0.20
on	preposition	1.0	0.05
others	…	small	~0

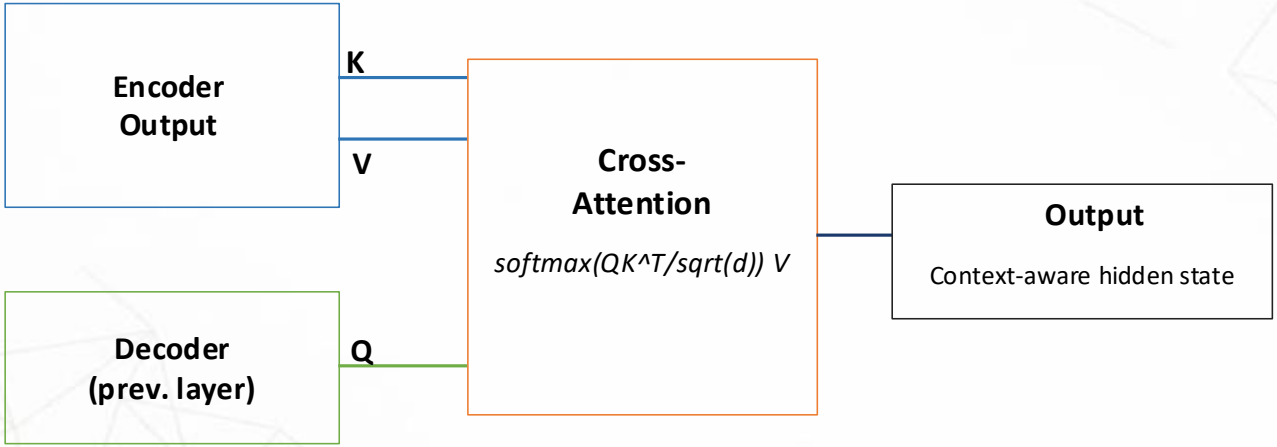
Output for "chat":

Output chat = **0.70** × Vcat + **0.20** × Vsat + …

"chat" becomes:
"blends cat + sat,
most weight on cat (subject)"

Cross-Attention: Encoder–Decoder Attention

Let the decoder ask questions of the encoder




```
Python
def cross_attention(Q, K, V):  # Q from decoder; K,V from encoder
    d_k = Q.shape[-1]
    scores = torch.bmm(Q, K.transpose(1, 2))  # (B, tgt_len, src_len)
    scores /= d_k **0.5                # scale to prevent vanishing grad
    weights = F.softmax(scores, dim=-1) # attention over src tokens
    output = torch.bmm(weights, V)  # weighted sum of values
    return output, weights
```

Cross-Attention — 3 Steps

Decoder query attends over encoder keys and values.

STEP 1

Similarity

$$score = Q_{dec} \cdot K_{enc}$$

Dot product of decoder query with each encoder key measures alignment.

STEP 2

Softmax

$$weights = softmax(scores / \sqrt{d_k})$$

Normalises alignment scores to a probability distribution over source tokens.

STEP 3

Weighted Sum

$$output = \Sigma (weights \times V_{enc})$$

Decoder token blends encoder values weighted by source-target alignment.

$$CrossAttention(Q, K, V) = softmax(\mathbf{QK^T} / \sqrt{d}) \cdot \mathbf{V}$$

Cross-Attention as Matrix Operations

Decoder queries attend over all encoder positions in parallel.

1 — Two Input Matrices $X_{enc} \in \mathbb{R}^{s \times d}$, $X_{dec} \in \mathbb{R}^{t \times d}$

s = src length, t = tgt length, d = embedding dim

2 — Project Q (decoder), K, V (encoder) $Q = X_{dec} \cdot W_Q$, $K = X_{enc} \cdot W_K$, $V = X_{enc} \cdot W_V$

Q from decoder; K, V from encoder

3 — Cross-Score Matrix $\text{Scores} = QK^T$ shape: t×s

Entry (i,j): how much decoder token i attends to encoder token j

4 — Scale + Softmax $A = \text{softmax}(QK^T / \sqrt{d_k})$

Each row: distribution over ALL encoder positions

5 — Output $\text{Output} = A \cdot V_{enc}$

Each decoder token: weighted blend of encoder values

Cross-Attention Matrix A (t × s)

	cat	sat	on	the	mat
cat	**0.60**	0.10	0.10	0.10	0.10
sat	0.10	**0.70**	0.05	0.05	0.10
on	0.10	0.10	**0.60**	0.10	0.10
the	0.05	**0.70**	0.05	0.05	0.15
mat	0.05	0.20	0.05	0.05	**0.65**

"chat" row → high attention on "cat" (source noun)

Tiny Cross-Attention — Step by Step

Encoder: x_1="cat", x_2="sat" · Decoder: y_1="chat" · d = 2

Encoder X_enc

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Q = Y·WQ (from decoder)

Scores = Q·K_enc^T shape: 1×2

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

softmax([1,0]) = [0.73, 0.27] ← "chat" attends more to "cat" (1×2 matrix — one decoder token over two encoder tokens)

WV (encoder weights)

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

K, V = X_enc·WK, X_enc·WV

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

→ **Attention A = softmax(Scores / √2)**

$$\begin{bmatrix} 0.73 & 0.27 \\ 0.27 & 0.73 \end{bmatrix}$$

Output = A · V_enc

"cat"

V_cat=[1,1], V_sat=[1,0]

A = [0.73, 0.27]

"chat"

Output = 0.73·[1,1] + 0.27·[1,0]

= [1.0, 0.73]

"chat" output [1.0, 0.73] → 2nd dim high, strongly influenced by "cat" (V=[1,1])

Cross-Attention: Decoder Reads the Encoder

✅ Cross-Attention

Q from DECODER
K, V from ENCODER
Bridges two separate sequences

❌ Self-Attention (not used here)

Q, K, V all from SAME sequence
Cannot bridge encoder → decoder
Needs two separate sequences

PyTorch — Cross-Attention Decoder Layer

```
class TransformerDecoderLayer(nn.Module):
    def __init__(self, d_model=512, n_heads=8, d_ff=2048, dropout=0.1):
        super().__init__()
        self.self_attn   = MultiHeadAttention(d_model, n_heads)
        self.cross_attn  = MultiHeadAttention(d_model, n_heads)  # cross-attn
        self.ffn         = nn.Sequential(
            nn.Linear(d_model, d_ff), nn.ReLU(), nn.Linear(d_ff, d_model))
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)

        self.norm3 = nn.LayerNorm(d_model)
    def forward(self, x, enc_out):
        # 1) Masked self-attention on decoder sequence
        x = self.norm1(x + self.self_attn(x, x, x))
        # 2) Cross-attention: Q from decoder, K+V from encoder!
        x = self.norm2(x + self.cross_attn(x, enc_out, enc_out))
        # 3) Feed-forward
        return self.norm3(x + self.ffn(x))
```

Cross-Attention: What It Can't Do

1 **Encoder context is fixed**

Cross-attention only lets the decoder look at encoder states. Within the encoder itself, each token is processed independently — no token can attend to other tokens in the same sequence.

```
# RNN Encoder: sequential, slow
for word in sentence:
    h = rnn_cell(word, h_prev)  # O(n) steps
```

2 **No position awareness**

Attention has no built-in notion of order. "The dog bit the man" and "The man bit the dog" look identical to naive attention.

```
# Attention is permutation-equivariant!
# shuffle tokens → same attention weights
```

3 **Separate Encoder & Decoder required**

For tasks like classification or language modeling, we don't need a separate encoder-decoder. We need one model that understands the full sequence at once.

```
# GPT/BERT don't use cross-attention at all
```

Self-Attention: Every Token Attends to All Others

❌ Cross-Attention

Q from decoder
K, V from encoder
Two separate sequences

✅ Self-Attention

Q, K, V all from SAME sequence
Every token looks at every other
One unified model

PyTorch — Self-Attention Block

```
class TransformerEncoderLayer(nn.Module):
    def __init__(self, d_model=512, n_heads=8, d_ff=2048, dropout=0.1):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, n_heads)
        self.ffn       = nn.Sequential(
            nn.Linear(d_model, d_ff), nn.ReLU(), nn.Linear(d_ff, d_model))
        self.norm1  = nn.LayerNorm(d_model)
        self.norm2  = nn.LayerNorm(d_model)
        self.drop   = nn.Dropout(dropout)

    def forward(self, x):
        # Self-attention: Q=K=V=x (same sequence!)
        attn_out = self.self_attn(x, x, x)
        x = self.norm1(x + self.drop(attn_out))  # residual + norm
        ffn_out = self.ffn(x)
        x = self.norm2(x + self.drop(ffn_out))   # residual + norm
        return x  # every token now has context from all other tokens
```

Self-Attention vs Cross-Attention

Same formula — different source of Q vs K, V. This single change transforms the mechanism completely.

	SELF-ATTENTION	CROSS-ATTENTION
Q source	Q = X·WQ (from same sequence)	Q = X_dec·WQ (from decoder)
K source	K = X·WK (from same sequence)	K = X_enc·WK (from encoder)
V source	V = X·WV (from same sequence)	V = X_enc·WV (from encoder)
Scores	Scores shape: n×n (src len × src len)	Scores shape: t×s (tgt len × src len)
Attn A	Each token attends to all OTHER tokens in same sequence	Each decoder token attends to ALL encoder tokens
Output	Output $\in \mathbb{R}^{n \times d}$ — same shape as input	Output $\in \mathbb{R}^{t \times d}$ — decoder shape, encoder content
Learns	Token understands itself in context of its own sequence	Decoder token understands itself via encoder context

When to use Self-Attention

Use inside the encoder (each token contextualises with all others) and inside the decoder (masked self-attn). Both Q, K, V come from the same sequence.

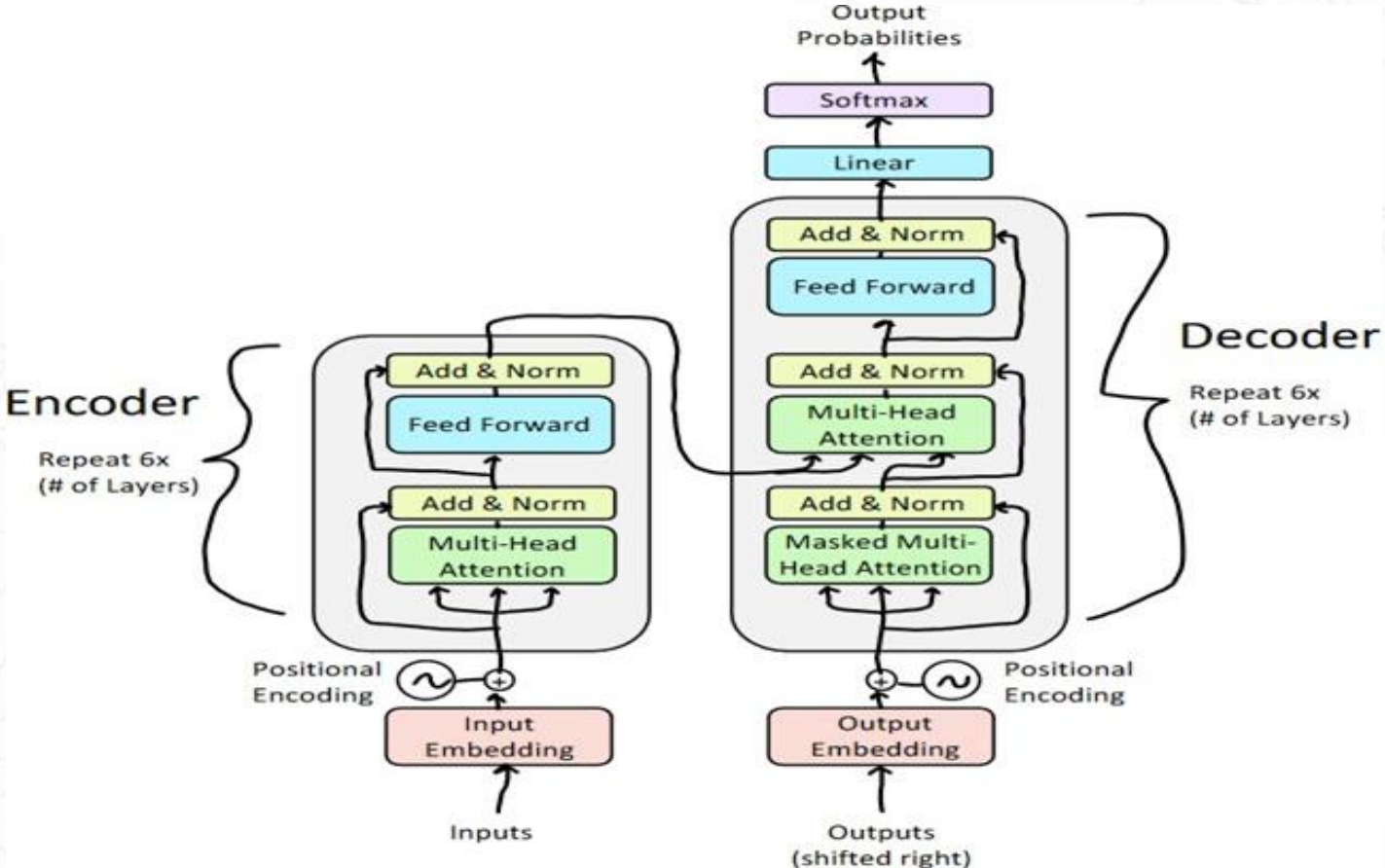
When to use Cross-Attention

Use in the decoder layers that need to read the encoder. Q comes from the decoder state; K and V come from the fixed encoder output.

Why the Transformer uses Both

The encoder uses self-attention to build rich source representations. The decoder uses masked self-attention then cross-attention to align and translate — each mechanism handles a distinct information flow.

The Transformer (Decoder)



The Complete Transformer Decoder — Summary

Masked Self-Attention

Causal mask: attend only to past tokens. Prevents future leakage during training.

Cross-Attention

Q from decoder, K/V from encoder. Dynamically reads source context.

Feed-Forward (FFN)

Two-layer MLP with ReLU, applied per token. d_ff = 4 x d_model.

Add & Norm

Residual + LayerNorm after every sub-layer. Stabilises training.

Positional Encoding

Sine/cosine vectors injected into embeddings. Order-aware attention.

Autoregressive Loop

One token per step. Append and repeat until <EOS> or max length.

Autoregressive Generation: The Decoder Loop

Start Token

Feed <BOS> as the first decoder input.

Forward Pass

Masked Self-Attn -> Cross-Attn -> FFN.

Linear + Softmax

Project to vocab; get probability dist.

Sample / Argmax

Greedy, beam search, or temperature sampling.

Append & Repeat

Append token; repeat from Step 2.

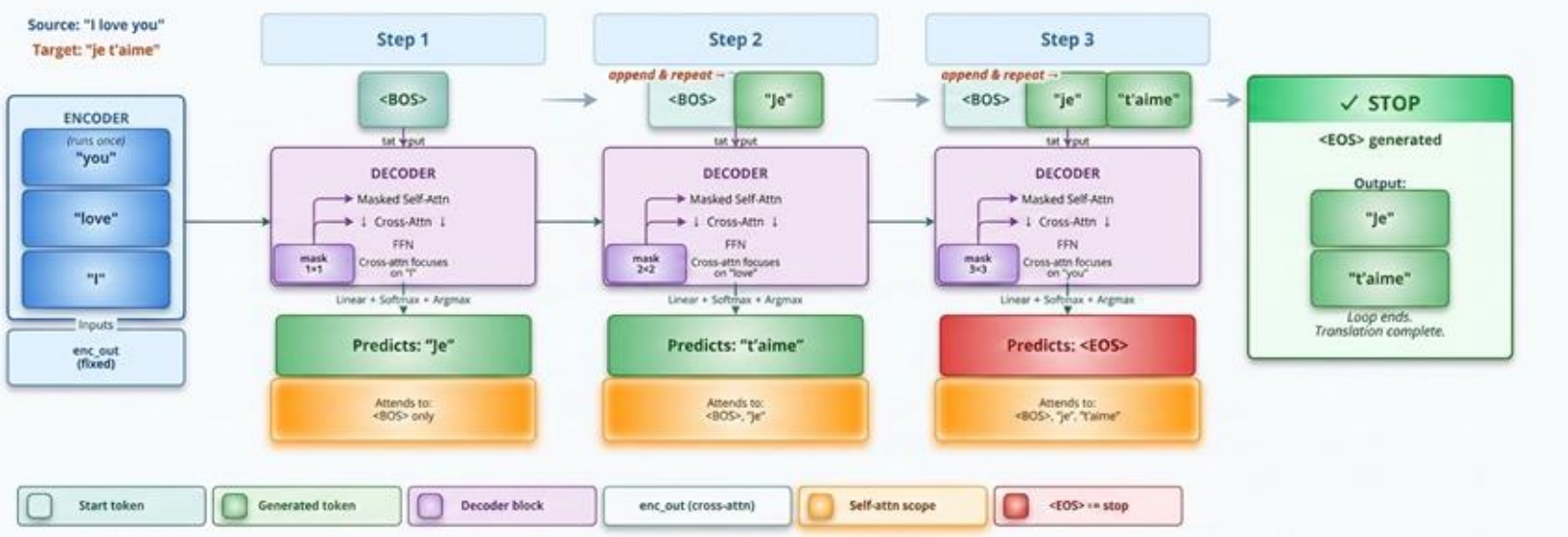
Stop Condition

Halt at <EOS> or max sequence length.

```
def generate(model, src, max_len=50, bos=1, eos=2):
    enc_out = model.encoder(src)          # encode source once
    tgt = torch.tensor([[bos]])               # start with <BOS>
    for _ in range(max_len):
        mask = causal_mask(tgt.size(1))    # rebuild mask each step
        logits = model.decoder(tgt, enc_out, mask)[:, -1, :]
        next_tok = logits.argmax(dim=-1).unsqueeze(0)  # greedy decode
        tgt = torch.cat([tgt, next_tok], dim=-1)  # append token
        if next_tok.item() == eos: break  # stop at <EOS>
    return tgt
```

Autoregressive Generation — Step by Step Example

Translating "I love you" → "Je t'aime" : how the decoder loop runs token by token



① Encoder runs ONCE
Source sentence is encoded into enc_out and held fixed. The decoder re-reads it at every step via cross-attention.

② Mask grows each step
Step 1: mask is 1×1. Step 2: 2×2. Step 3: 3×3. Rebuilt every iteration as the target sequence lengthens.

③ Append & repeat
Each predicted token is concatenated to the input and fed back in. The decoder sees its own growing output.

✓ <EOS> stops the loop
When the model predicts <EOS>, or max_len is reached, generation halts. Output is everything between <BOS> and <EOS>.

Quiz 1

In a Transformer model, what is the primary difference between **Self-Attention** and **Cross-Attention** in terms of where the "Queries" (**Q**), "Keys" (**K**), and "Values" (**V**) come from?

- **A)** In Self-Attention, **Q, K** and **V** all come from the same input sequence, whereas in Cross-Attention, **Q** comes from the decoder and **K** and **V** come from the encoder.
- **B)** In Self-Attention, **Q** and **K** come from the encoder, while in Cross-Attention, they come from the decoder.
- **C)** There is no difference; both mechanisms use the same sources for **Q, K** and **V**.
- **D)** Self-Attention is only used in the decoder, while Cross-Attention is only used in the encoder.

Quiz 1

In a Transformer model, what is the primary difference between **Self-Attention** and **Cross-Attention** in terms of where the "Queries" (**Q**), "Keys" (**K**), and "Values" (**V**) come from?

- ✔ **A)** In Self-Attention, **Q, K** and **V** all come from the same input sequence, whereas in Cross-Attention, **Q** comes from the decoder and **K** and **V** come from the encoder.
- **B)** In Self-Attention, **Q** and **K** come from the encoder, while in Cross-Attention, they come from the decoder.
- **C)** There is no difference; both mechanisms use the same sources for **Q, K** and **V**.
- **D)** Self-Attention is only used in the decoder, while Cross-Attention is only used in the encoder.

Explanation : Self-Attention allows a sequence to look at itself to find correlations between its own tokens. Cross-Attention is the "bridge" that allows the decoder to focus on relevant parts of the original input sequence (the encoder's output).

The Evolution of Representations

One-Hot	✓ Gave us structure	⚠ Sparse, no semantics
Dense Embed.	✓ Compact vectors	⚠ Same vector, all contexts
Word2Vec	✓ Semantic meaning	⚠ Static: bank = bank
ELMo/BiLSTM	✓ Contextual vectors	⚠ Sequential, slow, limited depth
Cross-Attention	✓ Alignment across seqs	⚠ No within-seq attention
Self-Attention	✓ Every token ↔ all	⚠ Position-agnostic
+ Pos. Encoding	✓ Order-aware! ✅	—

Transformers — Attention is All You Need (2017)

Self-Attention Mechanism

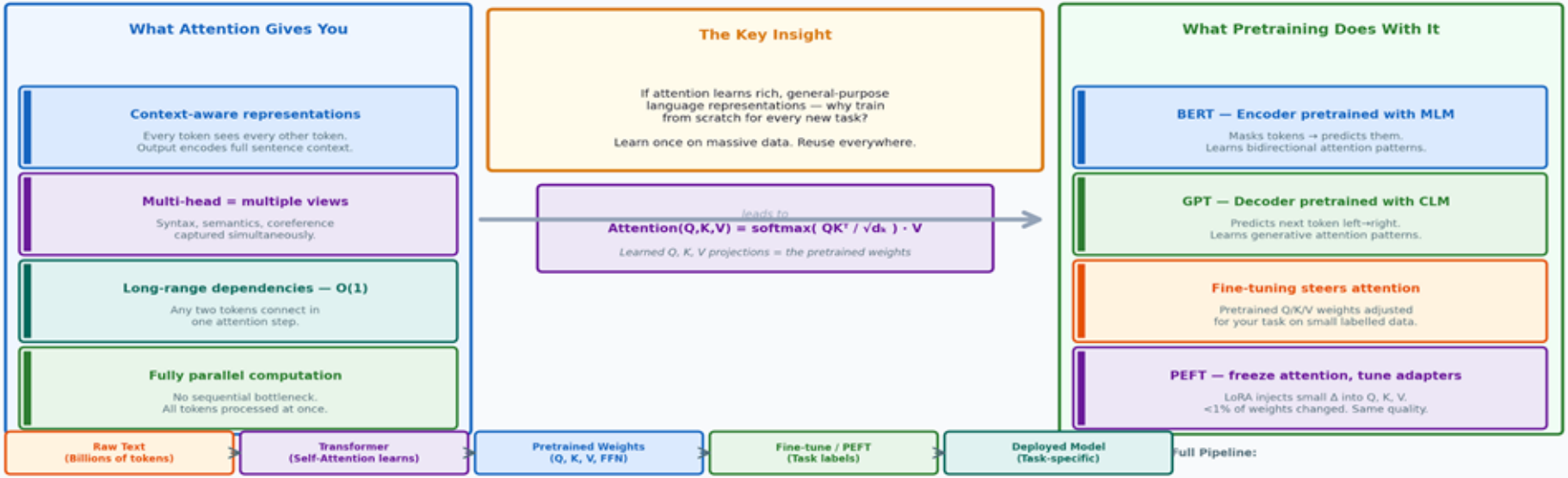
- Each token attends to ALL other tokens
- Query (Q), Key (K), Value (V) matrices
- $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$
- Captures long-range dependencies instantly
- Parallelizable (unlike RNNs — no sequential dependency)
- Multi-Head Attention: Run multiple attention heads in parallel, capture different relationship types

Why Transformers Changed Everything

- BERT (2018): Bi-directional encoder for NLP tasks — pre-train, fine-tune paradigm
- GPT series: Decoder-only for generative tasks (GPT-4, ChatGPT)
- Vision Transformer (ViT): Apply attention to image patches
- DALL-E, Stable Diffusion: Image generation
- AlphaFold 2: Protein structure prediction
- Scale: GPT-4 ~1.76 trillion parameters; emergent capabilities at scale

From Attention to Pretrained Models

Why the Transformer's attention mechanism makes pretraining not just possible — but inevitable



← Attention = general representations

Q, K, V weight matrices learn universal patterns of language — syntax, semantics, coreference — not task-specific features.

★ The key insight

General representations trained once on massive data transfer to any downstream task. Task-specific labels steer, not rebuild.

→ Pretraining = learning attention

BERT/GPT pretraining IS learning the Q, K, V projections. Fine-tuning adjusts them. PEFT injects tiny changes (LoRA Δ).

✓ Unified view

Every model from BERT to GPT-4 to LLaMA is the same idea: Transformer attention pretrained at scale, steered at fine-tune time.

The Modern Transformer Zoo

BERT

Encoder only

→ Classification, NER, QA

GPT-4

Decoder only

→ Generation, chat, code

T5 / BART

Encoder-Decoder

→ Translation, summarization

ViT

Vision Transformer

→ Image classification

Whisper

Speech → Text

→ ASR, multilingual

CLIP

Vision-Language

→ Image-text matching

All of these are just: Embeddings + Positional Encoding + Self-Attention + FFN, stacked.

Quiz 2

Why are **Positional Encodings** added to the input embeddings in a Transformer model?

- **A)** To reduce the total number of parameters in the model.
- **B)** To allow the model to process tokens sequentially like an RNN.
- **C)** To provide the model with information about the order or position of words, since the attention mechanism itself is permutation-invariant (it doesn't inherently know word order).
- **D)** To normalize the input data before it enters the Multi-Head Attention layer.

Quiz 2

Why are **Positional Encodings** added to the input embeddings in a Transformer model?

- **A)** To reduce the total number of parameters in the model.
- **B)** To allow the model to process tokens sequentially like an RNN.
- ✔ **C)** To provide the model with information about the order or position of words, since the attention mechanism itself is permutation-invariant (it doesn't inherently know word order).
- **D)** To normalize the input data before it enters the Multi-Head Attention layer.

Explanation : Unlike RNNs, which process words one by one, Transformers process all words in a sequence simultaneously. Without Positional Encodings, the model would treat the sentence "The cat chased the mouse" exactly the same as "The mouse chased the cat."

Thank you

Instructor



Niloy Ganguly
niloy@cse.iitkgp.ac.in

Teaching Assistants (IITKGP)



Sombit Bose



Shrimon Mukherjee